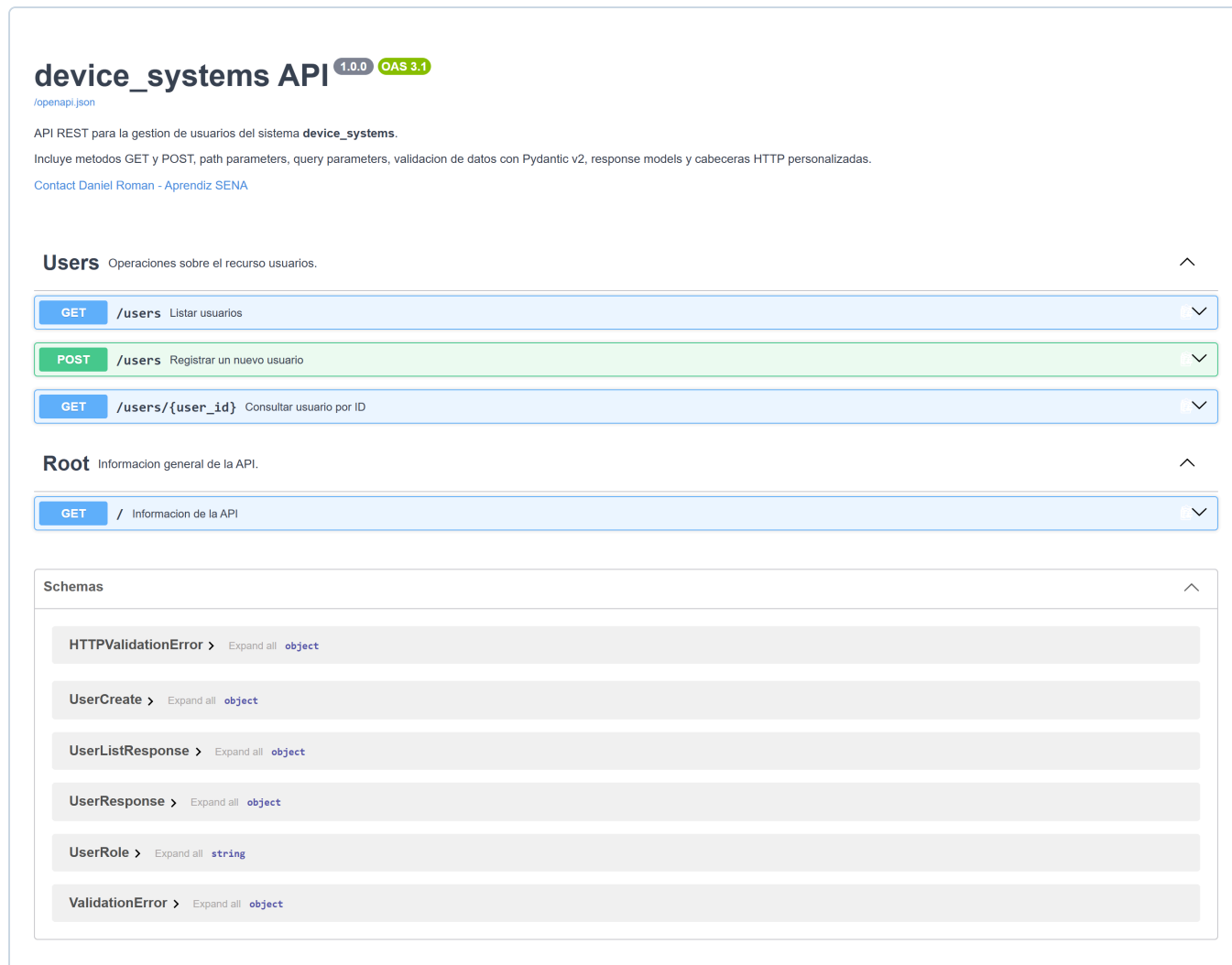


Evidencias EV07 — Fundamentos de FastAPI

Guia: GA1-220501096-01-AA1-EV07 **Aprendiz:** Daniel Roman **Proyecto:** device_systems (v1.0.0) — código en `../device_systems/` **Repositorio:** <https://github.com/daniel-roman345/python-avanzado->

1. Capturas de Swagger UI

Documentación automática generada por FastAPI en `/docs`, con los metadatos de la API (título, descripción, versión, contacto) y los endpoints agrupados por tags.



device_systems API 1.0.0 OAS 3.1
/openapi.json

API REST para la gestión de usuarios del sistema **device_systems**.
Incluye métodos GET y POST, path parameters, query parameters, validación de datos con Pydantic v2, response models y cabeceras HTTP personalizadas.
[Contact Daniel Roman - Aprendiz SENA](#)

Users Operaciones sobre el recurso usuarios. ^

- GET** `/users` Listar usuarios ▼
- POST** `/users` Registrar un nuevo usuario ▼
- GET** `/users/{user_id}` Consultar usuario por ID ▼

Root Información general de la API. ^

- GET** `/` Información de la API ▼

Schemas ^

- HTTPValidationError** Expand all object
- UserCreate** Expand all object
- UserListResponse** Expand all object
- UserResponse** Expand all object
- UserRole** Expand all string
- ValidationError** Expand all object

2. Evidencia de pruebas `GET /users`

Listado completo de usuarios. En la respuesta se observan el `total`, el arreglo `data` y las **cabeceras personalizadas** `x-app-name`, `x-api-version` y `x-total-users`.

GET

/users

Listar usuarios

Lista todos los usuarios registrados. Permite filtrar con query parameters por rol (**role**) y por estado (**is_active**).

Parameters

Cancel

Name	Description
role string (string null) (query)	Filtra por rol: admin, support o user --
is_active boolean (boolean null) (query)	Filtra por estado activo (true) o inactivo (false) --

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  http://127.0.0.1:8007/users/ \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8007/users

Server response

Code	Details
200	Response body <pre>{ "total": 3, "data": [{ "name": "Ana Perez", "email": "ana@sena.edu.co", "role": "admin", "is_active": true, "id": 1 }, { "name": "Carlos Gomez", "email": "carlos@sena.edu.co", "role": "support", "is_active": true, "id": 2 }, { "name": "Laura Martinez", "email": "laura@sena.edu.co", "role": "user", "is_active": false, "id": 3 }] }</pre> Response headers <pre>content-length: 293 content-type: application/json date: Sat, 12 Sep 2026 17:46:52 GMT server: uicorn x-api-version: 1.0 x-app-name: device_systems x-total-users: 3</pre>

Responses

Code	Description	Links
200	Listado de usuarios encontrados Media type application/json Controls Accept header Example Value Schema	No links
422	Validation Error Media type application/json Example Value Schema	No links

Filtro por query parameter `role=admin`

GET /users

Listar usuarios

Listar todos los usuarios registrados. Permite filtrar con query parameters por rol (`role`) y por estado (`is_active`).

Parameters

Cancel

Name	Description
role	Filtra por rol: admin, support o user
string (string null) (query)	admin
is_active	Filtra por estado activo (true) o inactivo (false)
boolean (boolean null) (query)	--

Execute

Clear

Responses

Curl

curl -X 'GET' \

'http://127.0.0.1:8007/users?role=admin' \

-H 'accept: application/json'

Request URL

http://127.0.0.1:8007/users?role=admin

Server response

Code	Details
200	Response body <pre>{ "total": 1, "data": [{ "name": "Ana Perez", "email": "ana@sena.edu.co", "role": "admin", "is_active": true, "id": 1 }] }</pre> Download Response headers content-length: 106 content-type: application/json date: Sat 12 Sep 2026 17:47:11 GMT server: uvicorn x-api-version: 1.0 x-app-name: device_systems x-total-users: 1

Responses

Code	Description	Links
200	Listado de usuarios encontrados Media type application/json Controls Accept header. Example Value Schema <pre>{ "total": 0, "data": [{ "name": "Ana Perez", "email": "ana@sena.edu.co", "role": "admin", "is_active": true, "id": 0 }] }</pre>	

 No links || 422 | Validation Error Media type application/json Example Value | Schema ``` { "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] } ``` | No links |

3. Evidencia de pruebas `GET /users/{user_id}`

Consulta por **path parameter**; devuelve `200 OK` con el usuario solicitado.

GET

/users/{user_id}

Consultar usuario por ID

Obtiene un usuario usando un path parameter.

Parameters

Cancel

Name	Description
user_id required integer (path)	Identificador del usuario

1

minimum: 1

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  http://127.0.0.1:8007/users/1' \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8007/users/1

Server response

Code	Details
200	Response body <pre>{ "name": "Ana Perez", "email": "ana@sena.edu.co", "role": "admin", "is_active": true, "id": 1 }</pre> Download Response headers <pre>content-length: 85 content-type: application/json date: Sat, 12 Sep 2026 17:46:55 GMT server: uvicorn x-api-version: 1.0 x-app-name: device_systems</pre>

Responses

Code	Description	Links
200	Usuario encontrado	No links
Media type application/json Controls Accept header. Example Value Schema <pre>{ "name": "Ana Perez", "email": "ana@sena.edu.co", "role": "admin", "is_active": true, "id": 0 }</pre>		
404	Usuario no encontrado	No links
422	Validation Error	No links
Media type application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] }</pre>		

4. Evidencia de pruebas POST /users

Registro de un usuario nuevo: responde 201 Created , devuelve el usuario con su id y agrega la cabecera location .

POST

/users Registrar un nuevo usuario

Crea un usuario validando los datos con Pydantic v2 y evitando correos duplicados.

Parameters

Cancel

Reset

No parameters

Request body required

application/json

Edit Value

Schema

```
{
  "name": "Pedro Ruiz",
  "email": "pedro@sena.edu.co",
  "role": "user",
  "is_active": true
}
```

Execute

Clear

Responses

Curl

curl -X 'POST' \
 'http://127.0.0.1:8007/users' \
 -H 'accept: application/json' \
 -H 'Content-Type: application/json' \
 -d '{
 "name": "Pedro Ruiz",
 "email": "pedro@sena.edu.co",
 "role": "user",
 "is_active": true
 }'

Request URL

http://127.0.0.1:8007/users

Server response

Code

Details

201

Response body

```
{
  "name": "Pedro Ruiz",
  "email": "pedro@sena.edu.co",
  "role": "user",
  "is_active": true,
  "id": 4
}
```

Download

Response headers

```
content-length: 87
content-type: application/json
date: Sat, 12 Sep 2026 17:47:02 GMT
location: /users/4
server: uvicorn
x-api-version: 1.0
x-app-name: device_systems
```

Responses

Code	Description	Links
201	Usuario creado correctamente	No links
Media type application/json Controls Accept header Example Value Schema<pre>{ "name": "Ana Perez", "email": "ana@sena.edu.co", "role": "admin", "is_active": true, "id": 0 }</pre>		
400	El correo ya esta registrado	No links
422	Datos invalidos	No links

Pagina 5 de 9

5. Evidencia de validaciones y errores

5.1 Usuario no encontrado — 404 Not Found

GET

/users/{user_id} Consultar usuario por ID

Obtiene un usuario usando un path parameter.

Parameters

Cancel

Name	Description
user_id * required integer (path)	Identificador del usuario

99

minimum: 1

Execute

Clear

Responses

Curl

curl -X 'GET' \
'http://127.0.0.1:8007/users/99' \
-H 'accept: application/json'

Request URL

http://127.0.0.1:8007/users/99

Server response

Code	Details
404	Error: Not Found

Response body

{
 "detail": "Usuario no encontrado"
}

Download

Response headers

content-length: 34
content-type: application/json
date: Sat, 12 Sep 2026 17:46:59 GMT
server: uvicorn
x-api-version: 1.0
x-app-name: device_systems

Responses

Code	Description	Links
200	Usuario encontrado	No links

Media type

application/json

Controls Accept header.

Example Value | Schema

{
 "name": "Ana Perez",
 "email": "ana@sena.edu.co",
 "role": "admin",
 "is_active": true,
 "id": 0
}

404

Usuario no encontrado

No links

422

Validation Error

No links

Media type

application/json

Example Value | Schema

{
 "detail": [
 {
 "loc": [
 "string",
 0
],
 "msg": "string",
 "type": "string",
 "input": "string",
 "ctx": {}
 }
]
}

5.2 Correo duplicado — 400 Bad Request

POST

/users

Registrar un nuevo usuario

Crea un usuario validando los datos con Pydantic v2 y evitando correos duplicados.

Parameters

Cancel

Reset

No parameters

Request body

required

application/json

Edit Value

Schema

```
{
  "name": "Pedro Ruiz",
  "email": "pedro@sena.edu.co",
  "role": "user",
  "is_active": true
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8007/users' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Pedro Ruiz",
    "email": "pedro@sena.edu.co",
    "role": "user",
    "is_active": true
  }'
```

Request URL

http://127.0.0.1:8007/users

Server response

Code	Details
400	Error: Bad Request Response body <pre>{ "detail": "El correo ya esta registrado" }</pre> Response headers <pre>content-length: 41 content-type: application/json date: Sat, 12 Sep 2026 17:47:04 GMT server: uvicorn x-api-version: 1.0 x-app-name: device_systems</pre>

Responses

Code	Description	Links
201	Usuario creado correctamente Media type application/json Controls Accept header. Example Value Schema <pre>{ "name": "Ana Perez", "email": "ana@sena.edu.co", "role": "admin", "is_active": true, "id": 0 }</pre>	No links
400	El correo ya esta registrado	No links
422	Datos invalidos	No links

5.3 Datos invalidos (Pydantic v2) — 422 Unprocessable Entity

Se envio `{"name": "ab", "email": "malo", "role": "root"}` y Pydantic reporto los tres errores: nombre corto, correo sin formato valido y rol fuera de los permitidos.

POST

/users

Registrar un nuevo usuario

Crea un usuario validando los datos con Pydantic v2 y evitando correos duplicados.

Parameters

Cancel

Reset

No parameters

Request body

required

application/json

Edit Value

Schema

{
 "name": "ab",
 "email": "malo",
 "role": "root",
 "is_active": true
}

Execute

Clear

Responses

Curl

```
curl -X 'POST' \  
  'http://127.0.0.1:8007/users' \  
  -H 'accept: application/json' \  
  -H 'Content-Type: application/json' \  
  -d '{  
    "name": "ab",  
    "email": "malo",  
    "role": "root",  
    "is_active": true  
  }'
```

Request URL

http://127.0.0.1:8007/users

Server response

Code

Details

422

Error: Unprocessable Content

Response body

```
{  
  "detail": [  
    {  
      "type": "string_too_short",  
      "loc": [  
        "body",  
        "name"  
      ],  
      "msg": "String should have at least 3 characters",  
      "input": "ab",  
      "ctx": {  
        "min_length": 3  
      }  
    },  
    {  
      "type": "value_error",  
      "loc": [  
        "body",  
        "email"  
      ],  
      "msg": "value is not a valid email address: An email address must have an @-sign.",  
      "input": "malo",  
      "ctx": {  
        "reason": "An email address must have an @-sign."  
      }  
    }  
  ],  
  "type": "enum",  
}
```

Download

Response headers

```
content-length: 503  
content-type: application/json  
date: Sat, 12 Sep 2026 17:47:07 GMT  
server: uvicorn  
x-api-version: 1.0  
x-app-name: device_systems
```

Responses

Code

Description

Links

201

Usuario creado correctamente

No links

Media type

application/json

Controls Accept header.

Example Value

Schema

{
 "name": "Ana Perez",
 "email": "ana@sena.edu.co",
 "role": "admin",
 "is_active": true,
 "id": 0
}

400

El correo ya esta registrado

No links

422

Datos invalidos

No links

6. Resumen de pruebas

#	Prueba	Esperado	Resultado
1	GET /users	200 + 3 usuarios	OK
2	GET /users?role=admin	200 + 1 usuario	OK
3	GET /users?is_active=false	200 + 1 usuario	OK
4	GET /users/1	200	OK
5	GET /users/99	404	OK
6	POST /users valido	201	OK
7	POST /users correo repetido	400	OK
8	POST /users datos invalidos	422	OK
9	Cabeceras X-App-Name / X-API-Version	Presentes	OK

7. Reflexion sobre el uso de FastAPI para construir APIs REST

Construir esta API me mostro que FastAPI resuelve con muy poco codigo tareas que normalmente son repetitivas. Al declarar los tipos en la firma de la funcion, el framework lee el path parameter, convierte el query parameter a `bool` o a `Enum`, valida el cuerpo de la peticion y devuelve un `422` con el detalle exacto del error, sin escribir validaciones manuales.

Pydantic v2 es la pieza central: separar `UserCreate` de `UserResponse` permite controlar que entra y que sale de la API. Ademas, los `response_model` alimentan la documentacion OpenAPI, de modo que Swagger UI sirve para probar y no solo para leer.

Por ultimo, el middleware de cabeceras `X-App-Name` y `X-API-Version` me mostro como interceptar todas las peticiones en un solo punto, algo clave para trazabilidad y versionado en las siguientes versiones de `device_systems`.